

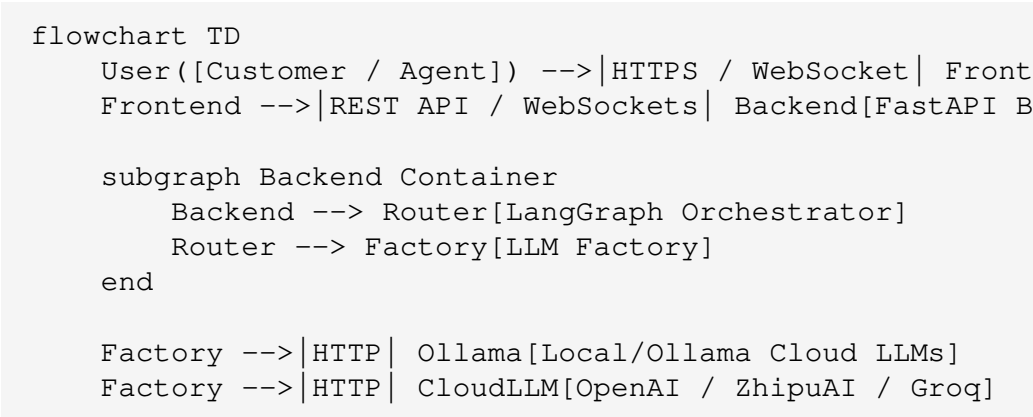
IntelliSupport AI (Project Bestie) - Technical Explainer

1. Executive Summary

IntelliSupport AI is an advanced, scalable, AI-powered customer support ecosystem. It acts as an intelligent intermediary between customers and businesses, providing instant, generative AI answers while seamlessly retrieving internal knowledge and customer records. When necessary, it routes complex or sensitive inquiries to human agents via real-time WebSocket communication and automated ticket creation.

2. Architecture Overview

The system follows a decoupled architecture, dividing responsibilities between a modern frontend and a robust, AI-orchestrating backend.



```
Router --> |RAG Queries| Pinecone[(Pinecone Vector DB)\nRouter --> |Async Cache| Redis[(Upstash Redis\nSession\nRouter --> |Transactional Queries| Supabase[(Supabase\n\nRouter --> |FastMCP| Gmail[Gmail Server\nAlerts & Noti\nRouter --> |FastMCP| WebSearch[Web Search Server]
```

Frontend (Next.js & React)

Hosted on Vercel, the frontend serves two main purposes:

- **Customer Chat Interface:** A responsive, real-time chat interface connecting to the AI backend via WebSockets.
- **Support Agent Dashboard:** An analytics and control panel for human agents to monitor active conversations, seamlessly take over escalated chats, and manage tickets and orders.

Backend (FastAPI & LangGraph)

Hosted via Docker, the backend is the central intelligence hub of the ecosystem:

- **LangGraph Orchestrator:** Uses a state machine pattern to intelligently route incoming messages based on intent and sentiment.
- **Multi-Provider LLM Factory:** Agnostically supports various models, allowing seamless switching between OpenAI, Anthropic, Google, and open-source models (via Ollama or vLLM).
- **FastMCP Integrations:** Implements the Model Context Protocol (MCP) to securely connect with external services like Gmail (for escalations/alerts) and Web Search.

3. Core Data Infrastructure

The ecosystem leverages specialized databases for distinct data types

to ensure optimal performance and security:

- **Vector Database (Pinecone):** Stores embeddings of knowledge base articles for RAG (Retrieval-Augmented Generation). This ensures the AI answers questions using accurate, company-specific information rather than hallucinating.
- **Relational Database (Supabase / PostgreSQL):** Stores structured business data, including customer profiles, past orders, and support tickets. The AI can execute safe transactional queries against this database to provide highly personalized, context-aware support (e.g., checking order status).
- **Session Store (Upstash Redis):** Manages persistent, high-performance conversational state. Because WebSockets require continuous state awareness, Redis enables asynchronous chat history retrieval and saving, allowing the AI to seamlessly recall past messages in any session.

```
sequenceDiagram
```

```
    participant C as Customer (Next.js)
    participant WS as FastAPI WebSocket
    participant L as LangGraph Engine
    participant R as Upstash Redis
```

```
C->>WS: Connect & Send Message
WS->>R: Async `get_session(session_id)`
R-->>WS: Return existing message history
WS->>L: `ainvoke()` with History + New Message
L-->>WS: Yield AI Tokens (Streaming)
WS-->>C: Stream tokens to UI
L->>R: Async `save_session(session_id)`
Note over R: State serialized to Cloud
```

4. The LangGraph State Machine Flow

The core innovation of the backend is its multi-agent orchestration.

When a customer sends a message, it enters the

`intent_router_node`, which dynamically determines the next best

step:

```
stateDiagram-v2
    [*] --> intent_router_node: Incoming Message

    state intent_router_node {
        [*] --> ClassifyIntent
        ClassifyIntent --> CheckSentiment
    }

    intent_router_node --> rag_node: intent == 'general'
    intent_router_node --> db_plan_node: intent == 'database'
    intent_router_node --> web_node: intent == 'web'
    intent_router_node --> escalation_node: sentiment == 'negative'

    state rag_node {
        [*] --> QueryPinecone
        QueryPinecone --> GenerateAnswer
    }

    state db_plan_node {
        [*] --> MapTools
        MapTools --> CheckRisk
    }

    db_plan_node --> db_execute_node: Risk == Low (Safe)
    db_plan_node --> human_approval_gate: Risk == High (Dangerous)

    human_approval_gate --> db_execute_node: WebSocket Approved
    human_approval_gate --> [*]: WebSocket Denied

    db_execute_node --> [*]: Return Result
    rag_node --> [*]: Return Answer
    web_node --> [*]: Return Result
    escalation_node --> [*]: Gmail Sent & Ticket Created
```

[!NOTE] Routing Paths in LangGraph:

1. **Classification & Sentiment Analysis:** Evaluates what the user

wants and their current emotional state.

2. **RAG Node** (`intent == 'general'`): Queries Pinecone and generates an answer using internal company knowledge.
3. **Database Node** (`intent == 'database'`): Maps tools to query Supabase. If the action is high-risk (e.g., destructive modifications), it enters a `human_approval_gate` where a human agent must explicitly approve the action via WebSocket.
4. **Web Search Node** (`intent == 'web'`): Searches the web for external information.
5. **Escalation Node** (`sentiment == 'negative'` OR **explicit request**): Bypasses the AI, sends a FastMCP Gmail alert, creates a support ticket in Supabase, and flags the session for immediate human takeover.

5. Security & Guardrails

- **Human-in-the-Loop (HITL)**: High-risk database operations are blocked pending human approval. The AI is structurally prevented from making unauthorized destructive changes.
- **Input/Output Guardrails**: The backend validates and sanitizes both user inputs and LLM outputs to prevent prompt injection and ensure responses remain on-brand, accurate, and safe.

6. Collaboration and Deployment

The project is container-native. The frontend is built for Vercel's edge network for maximum global responsiveness, while the backend is Dockerized to run on scalable container platforms (like Hugging Face Spaces or AWS).

This architecture allows businesses to scale their AI support effortlessly while maintaining complete control over their proprietary data, conversational state, and human-fallback mechanisms.